
Mentis3D: Reproduce Objects from Images with Code

Anton Landerer[†] Ephraim Shalunov[†] Jeremi Nuer[‡] Ryan Hung[†]
{alanderer, shalunov, jereminuer, rhung}@ucsb.edu
Departments of Computer Science[†], Electrical & Computer Engineering[‡]
University of California, Santa Barbara

Abstract

We present MENTIS3D, a fine-tuned model based on Qwen2.5-VL-3B-Instruct, that reconstructs 3D objects from 2D images by generating executable CadQuery Python code. Unlike existing image-to-3D methods that rely on specialized geometric tools, our approach generates human-interpretable procedural code, enabling unambiguous validation (using chamfer distance on pointclouds) and breakthrough interpretability. The motivation for our work rests in reasoning coherence. Successful programmatic reproduction of a 3D object from 2D views implies that the model "understands" the object's three-dimensional geometry, in some sense. MENTIS3D must, at some point, create an internal 3D representation from the sparse visual inputs it receives. Since our training data consists of procedurally generated arbitrary solids, our successful validations on real objects guarantee that MENTIS3D has generalized its geometric intuitions. We chose code generation as our evaluation domain because spatial understanding validated through executable code provides rigorous, objective metrics. MENTIS3D thus exhibits robust multi-domain reasoning spanning visual perception, internal spatial modeling, and structured code generation. We evaluate MENTIS3D on its characteristic object reconstruction task, demonstrating its ability to bridge the gap between 2D visual perception and 3D geometric understanding through interpretable code generation.

1 Introduction

Vision Language Models (VLMs) have rapidly emerged as powerful tools for the semantic understanding of visual information, achieving remarkable success in tasks such as image captioning, visual question answering, and scene description. However, despite their impressive semantic capabilities, these models still struggle significantly with spatial and geometric reasoning, a limitation particularly evident in applications requiring precise manipulation of 3D structures.

This limitation becomes glaringly apparent in Computer-Aided Design (CAD) tasks. Large Language Models (LLMs), known for their proficiency in generating executable code, often falter when asked to reason spatially about how that code translates into accurate 3D geometry. The disconnect between textual proficiency and spatial understanding highlights a fundamental challenge: while generating code is a strength, anticipating the spatial outcomes of that code remains a challenge for current models.

The task of reconstructing 3D objects from 2D images presents an ideal framework for exploring and enhancing the spatial reasoning capabilities of VLMs. Procedural code offers interpretable analysis and validation through quantifiable similarity metrics, such as chamfer distance between generated and ground-truth point clouds. This enables a dense reward signal and the use of Reinforcement Learning to improve spatial reasoning.

To address this gap, we introduce MENTIS3D, a 3-billion-parameter open-source Vision Language Model (VLM) fine-tuned from the Qwen2.5-VL-3B-Instruct architecture, designed specifically to re-

construct 3D objects by generating executable CadQuery Python code from visual input. Unlike prior research efforts that have explored geometric reasoning [14] or understanding CAD sequences [22], MENTIS3D uniquely demonstrates the ability to comprehensively interpret 2D visual representations and convert them directly into procedural, ready-to-use CAD code, opening avenues for automation and integration into practical workflows.

Our contributions are threefold:

- We create and curate a novel, augmented dataset comprising paired images and corresponding point clouds, derived and enriched from the cad-recode and Fusion360 datasets.
- We release fully trained model checkpoints, both after supervised fine-tuning (SFT) and after reinforcement learning (RL), to facilitate further research and exploration.
- We validate our model through rigorous evaluation metrics, clearly demonstrating the improvements in spatial reasoning and procedural CAD code generation capabilities.

2 Related Work

2.1 Large Language Models for Computer-Aided Design

Recent advancements have seen Large Language Models (LLMs) increasingly applied to Computer-Aided Design (CAD), primarily in data generation and parametric CAD creation. In data generation, LLMs have been leveraged to produce textual descriptions or captions of 2D renders derived from 3D objects, facilitating various downstream tasks [27, 24, 8, 19, 20].

Another significant area is parametric CAD generation, where LLMs generate parametric sequences—typically represented as structured JSON files—rather than executable code [23, 10, 24, 30, 13, 20].

A related study by [26] investigates the reverse-engineering of 3D CAD models from images using GPT4-V to infer a global base structure and extract semantic details, subsequently generating parametric CAD sequences for individual parts. Our approach simplifies this pipeline by performing object reconstruction in a single inference step, focusing explicitly on direct code generation rather than parametric sequences.

CAD code generation itself has been explored by [9], who examined inputs of varied modalities such as textual descriptions, images, sketches, and ground-truth 3D shapes. Other works like [18, 6] have fine-tuned LLMs specifically for CAD code generation but limited their inputs to purely textual data.

In contrast, [1] incorporated Vision Language Models (VLMs) indirectly: an LLM generates CAD code, with a separate VLM module verifying the visual output. Similarly, [2] employed VLMs for textual descriptions of intermediate renderings from LLM-generated CAD code. While related, these multi-step verification processes differ structurally from our more streamlined, single-step inference pipeline.

The work by [15] is particularly relevant, as it involves reverse-engineering 3D models by projecting point clouds into an embedding space utilized by an LLM to output CadQuery code. Their dataset, consisting of paired models and CadQuery ground-truth code, serves as a foundational resource for our supervised fine-tuning (SFT) phase.

2.2 General Spatial Reasoning with Vision Language Models

Vision Language Models (VLMs) have primarily been employed for semantic understanding tasks, including image captioning and visual question answering. However, there is increasing interest in evaluating and enhancing their geometric and spatial reasoning capabilities.

Several benchmarks have been established to assess these capabilities. Notably, datasets such as Euclid [28], MM-Math [17], GeoSense [25], MathVerse [29], and MathVista [11] evaluate VLM performance in geometric comprehension and mathematical reasoning tasks that incorporate visual components.

A relevant benchmark created by [12], GeoGramBench, specifically targets geometric reasoning capabilities involving procedural code generation. This benchmark aligns closely with our objective,

and investigates whether VLMs can effectively reason spatially about how procedural code translates into accurate 3D geometries.

However, our approach differs in that we use 3D CAD design as a benchmark for evaluating the spatial reasoning capabilities of our model, as motivated in the introduction.

3 Methodology

3.1 Problem Formulation

Let $\mathcal{C}$ denote the set of all valid CadQuery code strings. Let $\mathcal{I}$ denote the set of all images, and let $\mathcal{O}$ denote the set of all 3D objects. Towards our problem statement, define a mapping

$$R : \mathcal{O} \rightarrow \mathcal{P}(\mathcal{I})$$

that returns a set of 2D renders given a 3D object. We define another mapping

$$\psi : \mathcal{P}(\mathcal{I}) \rightarrow \mathcal{C}$$

such that $(\pi \circ R)(o)$ is CadQuery code string that best reconstructs $o \in \mathcal{O}$ when executed. The goal of MENTIS3D is to learn an approximation of ψ that we will call π .

3.2 CadQuery

We chose CadQuery in Python for 3D design due to the pre-existing capabilities of foundation models at writing Python code, and the very simple natural language-like structure of the library.

3.3 Dataset Creation

We construct our training dataset by augmenting the CAD-Recode dataset [15], which contains procedurally generated 3D objects paired with their corresponding CadQuery Python scripts. Starting with a subset of approximately 100,000 CadQuery scripts, we developed a comprehensive, multithreaded processing pipeline to generate multi-view rendered images and associated geometric representations suitable for both supervised fine-tuning and reinforcement learning.

Our data generation pipeline begins by compiling each CadQuery script into STL mesh objects. To enhance the geometric information conveyed in our renderings, we compute normal maps for each object, where the surface normal vectors at each point are encoded as RGB values, mapping the normal vector $\vec{n} = \langle x, y, z \rangle$ to a color $c_{\text{rgb}} = \langle r, g, b \rangle$ by normalizing and clipping $\{x, y, z\}$ and then assigning $r \leftarrow x, g \leftarrow y, b \leftarrow z$. This representation provides rich cues about surface geometry that complement the standard visual appearance, and give MENTIS3D a fighting chance to understand the objects it sees.

For each object, we capture multiple viewpoints by randomly selecting four views from a set of 14 canonical perspectives. These perspectives comprise the six orthogonal faces of the object’s bounding box (front, back, left, right, top, bottom) and eight diagonal views from the corners ($\{\text{above, below}\} \times \{\text{n, s}\} \times \{\text{e, w}\}$), as illustrated in Figure 1. This comprehensive set of potential viewpoints ensures that our model encounters diverse visual representations during training, improving its ability to reconstruct objects from various angles at test time. We also include labeled axes, to enable the model to reconstruct scale and orientation.

To maximize the geometric information density in our rendered images, we implement an optimized lighting configuration based on principles from optimal experimental design for photometric stereo. Our solution employs a 55-degree slanted, equi-azimuthal triad of point lights for each viewpoint, which maximizes the determinant of the Fisher information matrix for surface normal estimation [16, 5]. This configuration ensures that every surface patch is illuminated from a well-conditioned set of directions while creating informative shadow patterns that enhance depth perception and geometric understanding [4, 7]. The rendering process is executed in a multithreaded PyVista environment on NVIDIA A40 GPUs rented from RunPod, enabling efficient—albeit still slow—generation of our dataset.

In addition to the multi-view images, we generate surface point clouds for each object by uniformly sampling 8,192 points from the object’s surface. These point clouds serve as the ground truth

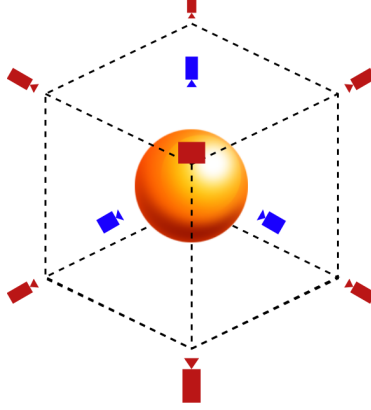


Figure 1: The 14 canonical viewpoints used for rendering 3D objects, showing the six orthogonal faces and eight diagonal perspectives of the bounding box. Some of the cameras are occluded.

representation for computing the Chamfer distance during reinforcement learning, providing a dense and unambiguous geometric signal for evaluating reconstruction quality.

Each entry in our final dataset consists of three components: the original CadQuery source code (used as ground truth for supervised fine-tuning), the surface point cloud (used for computing rewards during reinforcement learning), and the four rendered images with normal maps (used as input for both training phases). We maintain separate training splits for the SFT and RL phases to ensure proper evaluation and prevent overfitting. This careful data curation process yields a rich dataset that captures both the visual appearance and geometric structure of 3D objects, enabling our model to learn the complex mapping from 2D visual inputs to procedural 3D representations.

3.4 Training

We initialize our approximation π as the Qwen2.5-VL-3B-Instruct model [3]. We then go through two training stages. First, we use Supervised Fine-Tuning (SFT). We then apply Group Relative Policy Optimization (GRPO) training.

3.4.1 STAGE 1: SFT

We found that the base Qwen2.5-VL-3B-Instruct model cannot effectively write CadQuery code. This made it ineffective to directly begin with GRPO, as the reward signal was too sparse and the exploration too difficult. This left us no choice but to perform SFT as a warm-start. As it turned out, SFT was far more impactful on our final results than GRPO.

Our SFT dataset consists of 80,000 samples, each represented as tuple (q, a) . q denotes the input images and basic text instructions concatenated into a single prompt. Each sample has four input images randomly sampled from our 14 rendered views. a denotes the ground truth CadQuery code associated with the images in q . Our training objective maximizes the likelihood of generating a given q :

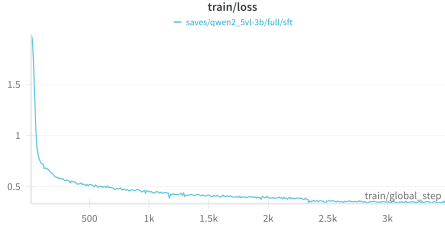
$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(q,a) \sim \mathcal{D}} \left[\sum_{t=1}^T \log p_{\theta}(a_t | q, a_{<t}) \right]$$

This is surprisingly effective. We only train for 3 epochs, but find it to be enough to achieve a validation loss below 0.4, as shown in Figure 2

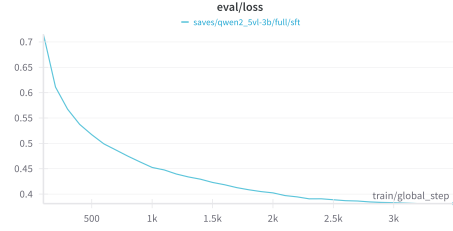
We show results post-SFT in Section 4.

3.4.2 STAGE 2: Reinforcement Learning through GRPO

The positive results from SFT allows us to continue with GRPO, as the reward signal becomes much denser, and the exploration problem becomes much less challenging.



(a) Training loss



(b) Validation loss

Figure 2: Loss curves for our SFT run on the recode dataset

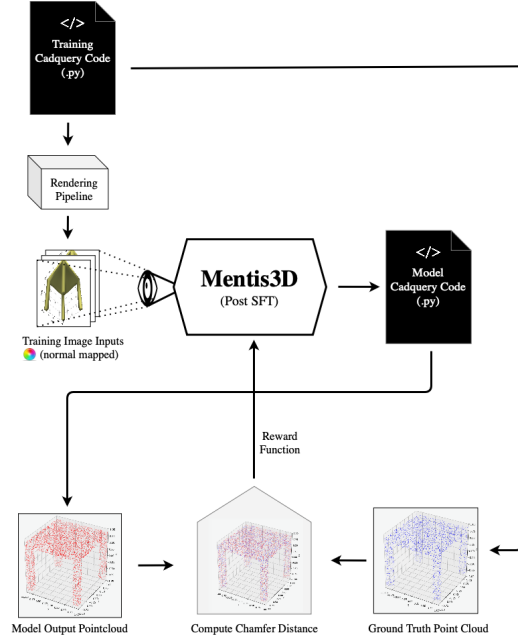


Figure 3: Pipeline for RL training: visual inputs, CadQuery prediction, pointcloud generation, and reward computation.

We continue to use our augmented CAD-Recode dataset for GRPO, at a smaller scale of 9000 samples compared to the 80,000 samples used for SFT. During GRPO, each sample in our dataset is a tuple (q, a) as before, but now a represents a ground truth point cloud of size n sampled from the target object.

Our reward function incentivizes valid code (whether it compiles) and the similarity of generated 3D models to those we are trying to recreate. This similarity is quantified by comparison of the pointclouds of these two objects through the *Chamfer Distance* metric.

Concretely,

$$CD(X, Y) = \frac{1}{|X|} \sum_{x \in X} \min_{y \in Y} \|x - y\|_2^2 + \frac{1}{|Y|} \sum_{y \in Y} \min_{x \in X} \|y - x\|_2^2$$

Define a mapping $f_{pc} : \mathcal{C} \rightarrow \mathbb{R}^{n \times 3}$ that returns a point cloud of size n given a valid CadQuery code string. Then we compute our reward as follows:

Algorithm 1 Reward Computation

```
1: Let  $\hat{c} = \pi(q)$ 
2: Let  $r_{\text{form}} = 0$  and  $r_{\text{acc}} = 0$ 
3: if  $\hat{c} \in \mathcal{C}$  then
4:    $\hat{a} = f_{pc}(\hat{c})$ 
5:    $r_{\text{form}} = 1$ 
6:    $r_{\text{acc}} = \exp(-\alpha \cdot \text{CD}(a, \hat{a}))$ 
7: end if
8: return  $0.1 \times r_{\text{form}} + 0.9 \times r_{\text{acc}}$ 
```

Notice that we assign some weight to format reward. We also require a hyperparameter α that determines how forgiving our accuracy reward is. In practice, we found that $\alpha = 40$ balances assigning higher reward to good outputs and lower reward to bad ones.

Let $\mathcal{D}$ be the prompt distribution. Each prompt $q \in \mathcal{D}$ is a concatenation of a set of four input renders and basic text instructions, as before. Let $\{o_i\}_{i=1}^G$ be a *group* of G sampled outputs. Let ρ_i be the importance ratio, and A_i be the *group-relative* advantage estimate. Advantage is calculated by normalizing the reward distribution of a group to have mean zero and standard deviation one. Let π_{ref} be the frozen reference policy, and, finally, let β be the KL-penalty coefficient. Then, we optimize the GRPO objective

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{q \sim \mathcal{D}} \left[\frac{1}{G} \sum_{i=1}^G \left(\min \left\{ \rho_i A_i, \text{clip}(\rho_i, 1 - \varepsilon, 1 + \varepsilon) A_i \right\} - \beta \mathbb{D}_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \right) \right]$$

$\{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$

where

$$\mathbb{D}_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(o_i | q)}{\pi_\theta(o_i | q)} - \log \frac{\pi_{\text{ref}}(o_i | q)}{\pi_\theta(o_i | q)} - 1$$
$$\rho_i = \frac{\pi_\theta(o_i | q)}{\pi_{\theta_{\text{old}}}(o_i | q)}$$

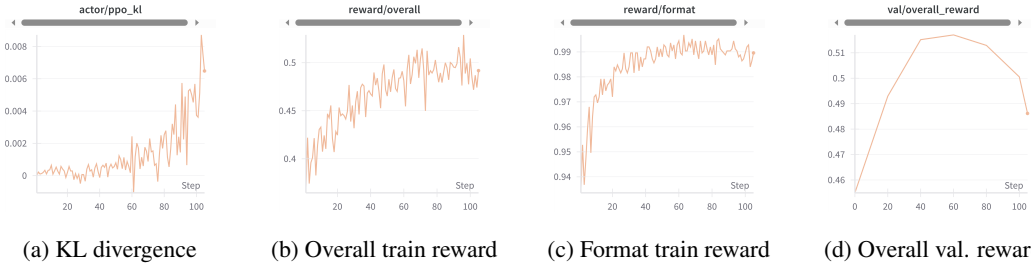


Figure 4: GRPO training process

We train for three epochs, but do early-stopping as our validation reward starts decreasing at around epoch 2.5, as seen in 4. This clearly corresponds to the KL divergence between our model and the reference policy destabilizing.

4 Results

In order to evaluate the performance of our model on CAD reconstruction tasks, we benchmark it against the Fusion360 Dataset [21], as many state-of-the-art models in this domain have been evaluated on this dataset.

We find that Mentis3D outperforms previous models on reconstruction tasks as measured by median and mean Chamfer Distance.

However, we note that Chamfer Distance is sensitive to the normalization strategy used when generating pointclouds from 3D objects. For this reason, we cannot confidently say that we have achieved state-of-the-art until the normalization strategy that other researchers use on this dataset is known to us and we can employ those same standards.

Method	Train Dataset		Fusion360 Test Set	
	Name	Size	Mean CD	Median CD
DeepCAD	DeepCAD	160k	330	89.2
PrismCAD	DeepCAD	127k	–	4.75
Point2Cyl	DeepCAD	35k	–	4.18
HNC-CAD	DeepCAD	125k	–	36.8
MultiCAD	DeepCAD	160k	–	42.2
TransCAD	DeepCAD	140k	78.6	33.4
CAD-Diffuser	DeepCAD	160k	–	3.85
CAD-SIGNet	DeepCAD	160k	7.37	0.48
CAD-Recode	DeepCAD	160k	3.37	0.52
CAD-Recode	CAD-Recode	1M	0.35	0.15
Mentis3D	Ours	90k	0.17	0.11

Table 1: Comparison of methods on the Fusion360 test set.

We perform several evaluation tests to compare the performance of the Qwen2.5VL 3B-SFT model before Reinforcement Learning and the final Mentis3D model after Reinforcement Learning.

We would like to understand how exactly RL affects the model’s ability to create 3D objects of varying complexity, and for this reason, we classify all examples in the Fusion360 dataset based on the number of intermediate "steps" required to create this object, as provided by the dataset. This can act as a proxy for the complexity of the CAD object we are trying to generate.

The performance of the MENTIS-SFT and MENTIS-RL models are evaluated by median CD, mean CD, and compilation rate.

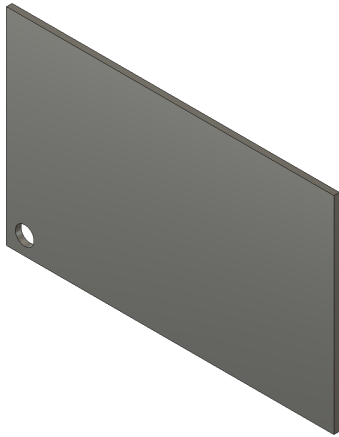
Inspecting the results from Table 2, we can draw an interesting insight: most of the accuracy gains as quantified by CD are achieved during SFT, with no meaningful improvement in the mean and median CD during RL.

However, we note that as the step (and therefore complexity of the CAD object) increases, the compilation rate of MENTIS-RL meaningfully improves over MENTIS-SFT. By step 5, there is a 16% absolute difference between the compilation rate of Mentis-RL and Mentis-SFT.

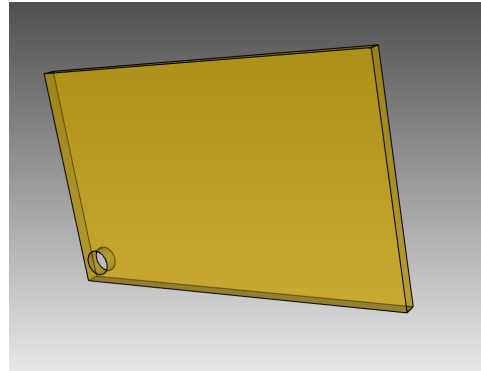
Thus, we can conclude that while SFT grounds the model in CadQuery syntax and basic visual understanding, Reinforcement Learning helps the model performance on more complicated objects.

Step	RL + SFT			SFT		
	Compilation Rate	Median CD	Mean CD	Compilation Rate	Median CD	Mean CD
1	0.94	0.10	0.16	0.89	0.10	0.16
2	0.95	0.13	0.18	0.87	0.12	0.17
3	0.94	0.13	0.18	0.86	0.12	0.16
4	0.93	0.12	0.17	0.86	0.12	0.18
5+	0.89	0.12	0.16	0.73	0.11	0.16
final	0.93	0.11	0.17	0.85	0.10	0.17

Table 2: Comparison of RL + SFT and SFT training: Compilation rate and Chamfer Distance (CD) statistics over steps.



(a) Ground truth CAD model



(b) Predicted CAD model

Figure 5: One example CAD model that our model predicted to a Chamfer Distance below 0.001

5 Conclusion and Future Directions

Overall, MENTIS3D has been shown to be extremely promising compared to similar models in the CAD generation space. Moreover, we take a more comprehensive image-to-code approach that promises more practical applications. While MENTIS3D’s results are encouraging for the most popular benchmarks, we believe that there is still room for improvement.

For example, a higher-quality dataset of human-verified images—alongside distortions, noise, and other variations—could lead to a more robust SFT outcome. The current training dataset was generated by language models, often resulting in confusing shapes that had very little physical sensibility.

Secondly, due to computational costs, this experiment was conducted with Qwen’s smallest 2.5-VL model with the least number of learnable parameters. With more resources, the same experiment can be replicated with even more powerful base models, likely generating more accurate outputs.

In the future, we hope to develop MENTIS3D into a turn-by-turn LLM tool that can design, iterate, and create alongside a human user. We hope to develop a modality of 3D modeling where a combination of sketches, natural language, and iterative corrections can yield an arbitrary object for a non-expert user. Furthermore, by extending our multimodal approach beyond images, we hope to find applications for the lessons we have learned in this project elsewhere in Large Language Model research.

6 Contributions

All authors contributed extensively, collaboratively, and creatively. The project took hundreds of person-hours. A few highlights: Landerer led the development of the reinforcement learning framework with chamfer distance rewards. Shalunov led dataset creation, developing the normal mapping technique encoding surface normals and mathematically optimized the lighting configuration across viewpoints. Nuer led supervised fine-tuning, establishing the training pipeline that enabled Qwen2.5-VL-3B-Instruct to generate executable CadQuery code. Hung conducted the continuous literature review throughout the work that guided us toward state-of-the-art results in direct image-to-code generation.

7 Impact Statement

We do not expect Mentis3D to have social implications; in particular, our model is far from capable of modeling human features or faces, and could not yet likely be used to generate any entertainment- or deception-quality assets.

8 Code

Our working project repository can be found at <https://github.com/echos-git/cs190a>. One of our datasets can be found at <https://github.com/echos-git/mentis-data-public>, while others can be found at <https://huggingface.co/shalunov>.

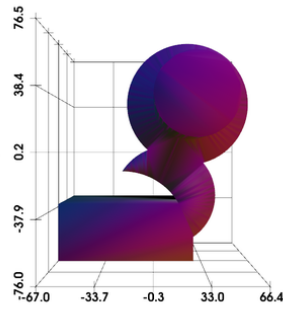
The MENTIS3D SFT checkpoint can be found on hugging face at https://huggingface.co/jereminuer/qwen25_vl_3b_sft and the RL checkpoint at <https://huggingface.co/afland/mentis-qwen2.5-vl-3b-grpo-step80>.

References

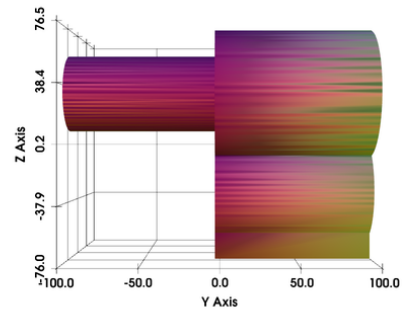
- [1] Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Zaidi, Megan Langwasser, Wei Xu, and Matthew Gombolay. Generating cad code with vision-language models for 3d designs, 2025. URL <https://arxiv.org/abs/2410.05340>.
- [2] Akshay Badagabettu, Sai Sravan Yarlagadda, and Amir Barati Farimani. Query2cad: Generating cad models using natural language queries, 2024. URL <https://arxiv.org/abs/2406.00144>.
- [3] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- [4] R. Basri and D. W. Jacobs. Lambertian reflectance and linear subspaces. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 25(2):218–233, 2003.
- [5] O. Drbohlav and M. J. Chantler. Can two specularly separated lights improve photometric stereo? *International Journal of Computer Vision*, 57(1):5–22, 2004.
- [6] Yuhao Du, Shunian Chen, Wenbo Zan, Peizhao Li, Mingxuan Wang, Dingjie Song, Bo Li, Yan Hu, and Benyou Wang. Blenderllm: Training large language models for computer-aided design with self-improvement, 2024. URL <https://arxiv.org/abs/2412.14203>.
- [7] J. Gardi, M. Collier, and M. J. Chantler. Optimal design of lighting in photometric stereo. *IEEE Transactions on Image Processing*, 30:1234–1249, 2021.
- [8] Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2cad: Generating sequential cad models from beginner-to-expert level text prompts, 2024. URL <https://arxiv.org/abs/2409.17106>.
- [9] Xingang Li, Yuewan Sun, and Zhenghui Sha. LLM4CAD: Multi-Modal Large Language Models for 3D Computer-Aided Design Generation. In *Proceedings of the ASME 2024 International Design Engineering Technical Conferences (IDETC)*, 2024. doi: 10.1115/DETC2024-143740.
- [10] Xueyang Li, Yu Song, Yunzhong Lou, and Xiangdong Zhou. Cad translator: An effective drive for text to 3d parametric computer-aided design generative modeling. In *Proceedings of the 32nd ACM International Conference on Multimedia*, MM ’24, page 8461–8470, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400706868. doi: 10.1145/3664647.3681549. URL <https://doi.org/10.1145/3664647.3681549>.
- [11] Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts, 2024. URL <https://arxiv.org/abs/2310.02255>.
- [12] Shixian Luo, Zezhou Zhu, Yu Yuan, Yuncheng Yang, Lianlei Shan, and Yong Wu. Geogrambench: Benchmarking the geometric program reasoning in modern llms, 2025. URL <https://arxiv.org/abs/2505.17653>.
- [13] Liane Makatura, Michael Foshey, Bohan Wang, Felix Hähnlein, Pingchuan Ma, Bolei Deng, Megan Tjandrasuwita, Andrew Spielberg, Crystal Elaine Owens, Peter Yichen Chen, Allan Zhao, Amy Zhu, Wil J Norton, Edward Gu, Joshua Jacob, Yifei Li, Adriana Schulz, and Wojciech Matusik. How can large language models help humans in design and manufacturing?, 2023. URL <https://arxiv.org/abs/2307.14377>.
- [14] Zhenyu Pan and Han Liu. Metaspatial: Reinforcing 3d spatial reasoning in vlms for the metaverse. *arXiv preprint arXiv:2503.18470*, 2025.
- [15] Danila Rukhovich et al. CAD-Recode: Reverse Engineering CAD Code from Point Clouds. *arXiv preprint arXiv:2412.14042*, 2024.

- [16] A. Spence and M. J. Chantler. Optimal photometric stereo light source placement. In *Proceedings of the British Machine Vision Conference (BMVC)*, 2001.
- [17] Kai Sun, Yushi Bai, Ji Qi, Lei Hou, and Juanzi Li. Mm-math: Advancing multimodal math evaluation with process evaluation and fine-grained classification, 2024. URL <https://arxiv.org/abs/2404.05091>.
- [18] Yuewan Sun, Xingang Li, and Zhenghui Sha. Large Language Models for Computer-Aided Design (LLM4CAD) Fine-Tuned: Dataset and Experiments. *Journal of Mechanical Design*, pages 1–19, 2025.
- [19] Ruiyu Wang, Yu Yuan, Shizhao Sun, and Jiang Bian. Text-to-cad generation through infusing visual feedback in large language models, 2025. URL <https://arxiv.org/abs/2501.19054>.
- [20] Siyu Wang, Cailian Chen, Xinyi Le, Qimin Xu, Lei Xu, Yanzhou Zhang, and Jie Yang. Cad-gpt: Synthesising cad construction sequence with spatial reasoning-enhanced multimodal llms. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(8):7880–7888, April 2025. ISSN 2159-5399. doi: 10.1609/aaai.v39i8.32849. URL <http://dx.doi.org/10.1609/aaai.v39i8.32849>.
- [21] Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences. *ACM Transactions on Graphics (TOG)*, 40(4), 2021.
- [22] Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A Deep Generative Network for Computer-Aided Design Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6776–6786, 2021.
- [23] Sifan Wu, Amir Khasahmadi, Mor Katz, Pradeep Kumar Jayaraman, Yewen Pu, Karl Willis, and Bang Liu. CAD-LLM: Large Language Model for CAD Generation. In *NeurIPS 2023 Workshop on Machine Learning for Creativity and Design*, 2023.
- [24] Jingwei Xu, Zibo Zhao, Chenyu Wang, Wen Liu, Yi Ma, and Shenghua Gao. CAD-MLLM: Unifying Multimodality-Conditioned CAD Generation With MLLM. *arXiv preprint arXiv:2411.04954*, 2024.
- [25] Liangyu Xu, Yingxiu Zhao, Jingyun Wang, Yingyao Wang, Bu Pi, Chen Wang, Mingliang Zhang, Jihao Gu, Xiang Li, Xiaoyong Zhu, Jun Song, and Bo Zheng. Geosense: Evaluating identification and application of geometric principles in multimodal reasoning, 2025. URL <https://arxiv.org/abs/2504.12597>.
- [26] Yang You, Mikaela Angelina Uy, Jiaqi Han, Rahul Thomas, Haotong Zhang, Suyu You, and Leonidas Guibas. Img2cad: Reverse engineering 3d cad models from images through vlm-assisted conditional factorization, 2024. URL <https://arxiv.org/abs/2408.01437>.
- [27] Zhe Yuan, Jianqi Shi, and Yanhong Huang. OpenECAD: An efficient visual language model for editable 3D-CAD design. *Computers & Graphics*, 124:104048, 2024. doi: 10.1016/j.cag.2024.104048.
- [28] Jiarui Zhang, Ollie Liu, Tianyu Yu, Jinyi Hu, and Willie Neiswanger. Euclid: Supercharging multimodal llms with synthetic high-fidelity visual descriptions, 2024. URL <https://arxiv.org/abs/2412.08737>.
- [29] Renrui Zhang, Dongzhi Jiang, Yichi Zhang, Haokun Lin, Ziyu Guo, Pengshuo Qiu, Aojun Zhou, Pan Lu, Kai-Wei Chang, Peng Gao, and Hongsheng Li. Mathverse: Does your multi-modal llm truly see the diagrams in visual math problems?, 2024. URL <https://arxiv.org/abs/2403.14624>.
- [30] Zhanwei Zhang, Shizhao Sun, Wenxiao Wang, Deng Cai, and Jiang Bian. Flexcad: Unified and versatile controllable cad generation with fine-tuned large language models, 2025. URL <https://arxiv.org/abs/2411.05823>.

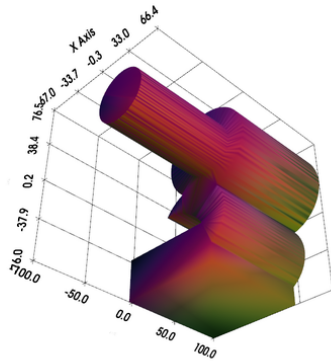
A Example Training Data



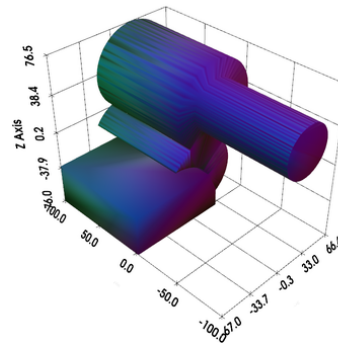
(a) Front view



(b) Left view



(c) Below_se view



(d) Above_sw view

Figure 6: Object 50024 (randomly chosen), four different views.